

IoT 入門圖解 QA

用考試用智慧停車場與自動停車小車，理解 ESP32-S3、感測器、致動器、3.3V / 5V、MQTT / HTTP、dashboard 與後續延伸。

適用課程：IoT 玩具與互動硬體設計

版本：2026-05-14

先記住這張大圖

現實世界

考試場、邊界、車位、小車、閘門、管理者

->

資料來源

碰撞開關、距離感測、按鈕、狀態事件

->

系統判斷

規則、計時、後端 API、MQTT topic

->

實體反應

ESP32-S3 控制燈號、蜂鳴器、伺服馬達

1. 板子與角色

ESP32-S3、Arduino、Raspberry Pi 到底差在哪。

2. 電路安全

GPIO、GND、3.3V / 5V、限流電阻、麵包板。

3. 元件分類

感測器、致動器、模組、開發板怎麼分。

4. 網路通訊

HTTP、MQTT、topic、message、dashboard。

5. 專題資料流

考試用智慧停車場、自動停車小車完整流程。

6. 延伸方向

自走車、夾娃娃機、ROS 2 怎麼接上來。

1. ESP32-S3、Arduino、Raspberry Pi 是什麼關係？

一句話：ESP32-S3 是本課程的「小型連網控制器」，Arduino 是入門寫法，Raspberry Pi 是比較像小電腦的 Linux 主機。

ESP32-S3

像一個很會控制現場設備的小腦袋。它便宜、可連 Wi-Fi、可接感測器、可控制馬達與 LED。

課程用途：閘門、燈號、蜂鳴器、教室感測器。

Arduino

在本課程多半指「Arduino framework」，也就是一套容易上手的寫法，不一定是 Arduino Uno 那片板子。

課程用途：用 Arduino IDE 寫 ESP32-S3。

Raspberry Pi

像一台小型 Linux 電腦，可以跑 Python、OpenCV、Web server、ROS 2，但成本與維護負擔較高。

課程用途：選修或期末進階。

三者的定位圖

Arduino 寫法

讓初學者容易寫 GPIO、Wi-Fi、馬達

->

ESP32-S3 板子

實際接線、讀感測器、控制實體裝置

+

筆電 / 手機

跑 OpenCV、dashboard、後端、資料庫

● 本課程必用 ● 軟體平台 ● 進階選修

問題	學生容易誤會	正確想法
ESP32-S3 可以跑 ROS 2 嗎？	以為買 ESP32-S3 就能完整跑 SLAM、Nav2。	ESP32-S3 可當 ROS 2 系統的低階控制器，但完整 ROS 2 主機通常要筆電或 Raspberry Pi。
為什麼不一開始買 Raspberry Pi？	以為越強越好。	本課程先練 IoT 資料流與實體控制。手機和筆電已能負責影像與後端，ESP32-S3 負責現場控制，成本最低。

課程例子：手機拍停車場，筆電跑 OpenCV 判斷車位，ESP32-S3 控制閘門與燈號。這樣不用 Raspberry Pi，也能完成完整 IoT 系統。

2. GPIO 是什麼？GND 又是什麼？

GPIO：General Purpose Input / Output，通用輸入輸出腳位。它是 ESP32-S3 跟外部元件溝通的腳。

GND：Ground，接地。它是整個電路共同的 0V 參考點。

GPIO 可以是輸入，也可以是輸出



輸入範例

按鈕被按下，GPIO 讀到 HIGH 或 LOW。

```
// PIN_BUTTON 來自教師公布的 verified
profile
int button = digitalRead(PIN_BUTTON);
if (button == LOW) {
  Serial.println("button pressed");
}
```

輸出範例

ESP32-S3 控制 LED 亮或暗。

```
// PIN_TEST_OUTPUT 來自教師公布的 verified
profile
pinMode(PIN_TEST_OUTPUT, OUTPUT);
digitalWrite(PIN_TEST_OUTPUT, HIGH); //
亮
delay(500);
digitalWrite(PIN_TEST_OUTPUT, LOW); //
暗
```

GND 的生活比喻

如果電壓像高度，GND 就是地面。大家要先約定同一個地面高度，才知道 3.3V、5V 到底是相對於哪裡。

ESP32-S3 與外部模組要共地

ESP32-S3 GND

控制訊號的參考點

連在一起

伺服 / 感測器 GND

模組讀懂訊號的參考點

常見錯誤：伺服馬達另外供電，但沒有把伺服 GND 和 ESP32-S3 GND 接在一起，結果伺服亂抖或完全不動。

接線口訣：先找 VCC、GND、Signal。VCC 供電，GND 共地，Signal 接 GPIO。不要看到三根線就亂插。

3. 3.3V / 5V 為什麼重要？可以統一嗎？

一句話：不同晶片和模組的安全工作電壓不同。ESP32-S3 的 GPIO 是 3.3V 邏輯，很多傳統 Arduino 模組是 5V 供電或 5V 訊號。

3.3V 常見位置

ESP32-S3 GPIO 訊號、高階感測器、I2C 模組、ESP32 本身邏輯。

安全原則 進入 ESP32-S3 GPIO 的訊號最好是 0V 到 3.3V。

5V 常見位置

USB 供電、SG90 伺服、HC-SR04 VCC、部分 LED / 蜂鳴器模組。

風險 5V 訊號直接進 ESP32-S3 GPIO，可能傷板子。

元件	供電可能用	訊號要注意
ESP32-S3	USB 5V 進板子，再由板上穩壓	GPIO 是 3.3V 邏輯
SG90 伺服	通常 5V 較穩	控制線可由 ESP32 GPIO 輸出 PWM，但要共地
HC-SR04	常見 VCC 5V	Echo 常是 5V，回 ESP32 前要降壓
DHT11 模組	多數可 3.3V 或 5V，依模組規格	用 3.3V 供電最省事

限流電阻是什麼？

限流電阻：用來限制電流，不讓 LED 或 GPIO 承受過大電流。可以想成水管中的縮口，讓水不要一下子衝太大。

LED 需要限流電阻

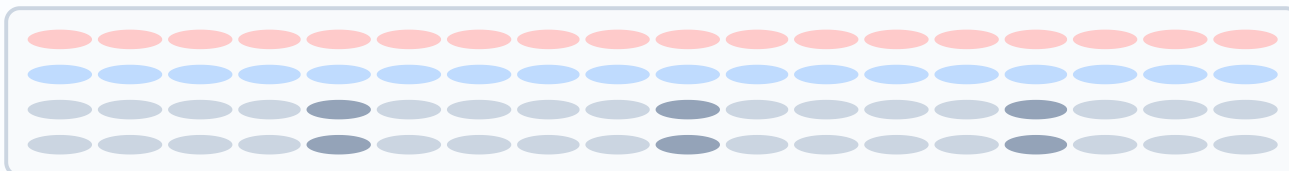


不統一的原因：電子元件不是同一世代、同一用途。低電壓省電且適合新晶片，高一點的 5V 對馬達、伺服、傳統模組比較常見。課堂重點不是背所有規格，而是每次接線前確認：供電電壓、訊號電壓、GND。

4. 麵包板和杜邦線怎麼接才不會燒掉？

麵包板：不用焊接就能暫時接電路的板子。**杜邦線**：用來把開發板、麵包板和模組接起來的線。

麵包板的洞不是每個都互通



- 紅色電源排通常接 3.3V 或 5V，只能選一種並貼標籤
- 藍色電源排通常接 GND
- 中間區域同一排互通，左右中間通常斷開

安全接線順序

1. 先拔 USB，不要帶電插拔。
2. 找出模組 VCC、GND、Signal。
3. 先接 GND，再接 VCC，最後接 Signal。
4. 拍照或畫接線圖，再上電測試。

危險行為

- 把 5V 直接接到 GPIO。
- 把 3.3V 和 GND 短路。
- 伺服馬達從 ESP32-S3 的 3.3V 腳吃電。
- 不知道模組腳位就直接插。

HC-SR04 Echo 為什麼要分壓？

常見 HC-SR04 的 Echo 腳會輸出 5V。ESP32-S3 GPIO 只適合 3.3V，因此 Echo 回到 ESP32-S3 前要降壓。

用 1k + 2.2k 做簡易分壓



提醒：這不是為了讓超音波更準，而是為了保護 ESP32-S3 腳位。

5. 感測器、致動器、模組、開發板怎麼分？

一句話：感測器把現實變成資料，致動器把指令變成動作，模組是已經整理好的小電路板，開發板是可寫程式的主控。

類別	像什麼	本課程例子	資料方向
開發板	小腦袋	ESP32-S3 DevKit	讀資料，也下指令
感測器	眼睛、耳朵、皮膚	DHT11、PIR、光敏、HC-SR04	現實 -> 數位資料
致動器	手、嘴巴、燈號	SG90、RGB LED、蜂鳴器	數位指令 -> 實體反應
模組	包裝好的零件小板	KY-016、KY-012、DHT11 模組	讓初學者比較好接

停車場中各元件的角色

手機相機

感測器，看到車位

->

筆電 OpenCV

判斷 A1 空/滿

->

ESP32-S3

收到結果，控制現場

->

SG90 / LED / 蜂

鳴器

致動器，做出反應

學生範例：如果你做「自動停車小車」，HC-SR04 是感測器，L298N 是馬達驅動模組，TT 馬達是致動器，ESP32-S3 是開發板。

6. HTTP、MQTT、Dashboard 是什麼？

Dashboard：不是硬體，也不是麵包板。它是使用者看到系統狀態的網頁或畫面，例如車位空滿、教室是否有人、異常告警。

HTTP 像填表單

裝置或網頁主動去呼叫 API。

```
POST /api/parking/A1
{
  "status": "occupied"
}
```

適合 dashboard 查詢、按按鈕控制、送一次資料。

MQTT 像訂閱頻道

大家都連到 broker，發布或訂閱 topic。

```
topic: parking/A1/status
message: occupied
```

適合多裝置、即時狀態、事件推播。

MQTT 的三個角色

Publisher

發布訊息
OpenCV、ESP32-S3

->

Broker

訊息中繼站
Mosquitto、EMQX

->

Subscriber

訂閱訊息
dashboard、ESP32-S3

本課程建議 topic 命名

topic	誰發	誰收	用途
parking/A1/status	OpenCV 或後端	dashboard	A1 車位空/滿
parking/gate/command	dashboard 或後端	ESP32-S3	命令閘門 open / close
classroom/room1/sensors	ESP32-S3	後端、dashboard	溫濕度、亮度、PIR 狀態

範例 JSON：

```
{
  "room": "A301",
  "motion": true,
  "light": 780,
  "temperature": 26.8,
  "humidity": 62,
  "time": "2026-05-14T10:30:00"
}
```

7. 手機視覺、ESP32-S3、Dashboard 彼此怎麼溝通？

一句話：手機負責拍，筆電負責看懂影像，ESP32-S3 負責控制現場，dashboard 負責讓人看懂狀態。

智慧停車場完整資料流



一步一步發生什麼事？

步驟	發生的事	資料例子
1	手機拍到停車場模型。	一張影像 frame
2	OpenCV 只看指定 ROI 車格。	A1、A2、A3 的矩形範圍
3	程式判斷車格空/滿。	A1=occupied
4	後端更新資料庫與 dashboard。	剩餘空位 2 格
5	ESP32-S3 收到命令並控制硬體。	空位可進場 -> 閘門開

學生最低可做版本：手機拍 3 個車位，OpenCV 用顏色或輪廓判斷空/滿，dashboard 顯示 3 格狀態，ESP32-S3 用 RGB LED 表示「可停 / 已滿」。

為什麼不用相機板？相機板是接在開發板上的小型攝影模組，例如 ESP32-CAM 或 Raspberry Pi Camera。它會增加硬體與驅動問題。初學者先用自己的手機，影像品質高、架設簡單、成本為 0。

8. 智慧停車場的完整資料流怎麼跑？

目標：把倒車入庫小車放回可管理的停車場，讓車輛、閘門、車位與 dashboard 使用同一套事件資料。

智慧停車場資料流



事件資料範例

event	slot_id	device_id	系統判斷
car_parked	A01	car_03	車位已占用
gate_open	入口	gate_01	閘門開啟
parking_error	A01	car_03	入庫失敗或卡住
slot_empty	A01	slot_sensor_01	車位已釋出

ESP32-S3 上傳資料範例：

```
{
  "lot_id": "lot_a",
  "slot_id": "A01",
  "event": "car_parked",
  "device_id": "car_03",
  "status": "occupied",
  "time": "2026-11-18T14:32:00+08:00"
}
```

這個專題在教什麼？它不是只讓車子會動，而是讓學生練習「實體事件 -> 裝置回報 -> dashboard 顯示 -> log 與異常處理」的完整 IoT 思維。

9. 後面怎麼延伸到自走車、夾娃娃機、ROS 2 ?

一句話：前半學期買的材料不是一次性玩具，而是後續專題的控制核心。差別在於後面加上新的機構、馬達、感測器或上位系統。

延伸路線圖

共同基礎

ESP32-S3、Wi-Fi、GPIO、MQTT、dashboard

->

互動機構

伺服、按鈕、LED、蜂鳴器

->

移動或機台

自走車、夾娃娃機、置物櫃、入口閘門

->

進階系統

ROS 2、Nav2、Gazebo、PX4、ArduPilot

想做什麼	沿用材料	可能加買	學到什麼
自走車	ESP32-S3、超音波、LED、蜂鳴器、dashboard	TT 馬達、馬達驅動板、底盤、電池盒	馬達控制、避障、任務狀態回報
夾娃娃機	ESP32-S3、SG90、按鈕、LED、蜂鳴器	更多伺服、搖桿、紙板或滑軌材料	機構控制、使用者互動、遊戲流程
ROS 2	ESP32-S3 當低階控制器	筆電或 Raspberry Pi、馬達編碼器、深度相機或 LiDAR	節點、topic、導航、模擬、機器人架構

ROS 2 需要深入嗎？

對這門課來說，ROS 2 不必成為全班期中前的必修深水區。它適合放在後半學期做「進階選修路線」：先讓學生知道 ROS 2 是大型機器人系統的資料流與模組化架構，再讓有興趣的組別接 Gazebo、Nav2、PX4 或 ArduPilot。

本課程的核心不是某一片板子，也不是某一台車。核心是：把便宜硬體、程式、資料、dashboard、流程與場域問題整合成能被人使用的 IoT 系統。

第 13-18 週節奏：第 13 週進行第二次個人筆試，不安排新進度；第 14 週完成自動反應、故障復原與乾淨環境重建；第 15 週進行第二次專題報告，並完成修正、彩排與版本凍結；第 16 至 17 週分批進行第三次專題報告；期末展示與個人問答，不安排新進度；第 18 週是校定期末考週，保留空白。

10. 如果分量太大，可以怎麼切成每週講義？

這份 PDF 可以先作為完整學生手冊。若課堂上覺得一次給太多，可以從同一份內容拆成週次講義。

週次	建議拆分主題	學生要能回答
第 1 週	課程大圖、ESP32-S3 / Arduino / Raspberry Pi 差異	為什麼這門課先用 ESP32-S3 和手機視覺？
第 2 週	ESP32-S3 開發環境、Serial、Wi-Fi scan	沒有完整材料包時，如何先確認板子能工作？
第 3 週	GPIO、GND、3.3V / 5V、麵包板安全	怎麼接線比較不會燒板子？
第 4-6 週	考試用智慧停車場、碰撞偵測、蜂鳴器、燈號	怎麼讓場地知道車子碰到邊界，並記錄違規？
第 7-9 週	自製小車、倒車入庫、測試紀錄、期中評分	如何讓車子不是靠運氣，而是在考場中可重複地倒車入庫？
第 10-12 週	考後檢討、期末題目、MQTT / WebSocket、dashboard	如何把考試經驗延伸成自己的 IoT project？
第 13-18 週	第二次筆試、故障復原、版本凍結、第二與第三次專題報告	如何理解完整軟硬架構，並以端到端證據完成第 15 與第 16-17 週報告？

學生檢核清單

接線前：我知道 VCC、GND、Signal 在哪裡。

上電前：我確認沒有把 5V 接進 ESP32 GPIO。

測試時：我一次只加一個元件，成功再加下一個。

寫程式前：我知道資料從哪裡來，要送到哪裡。

做 dashboard 前：我知道使用者需要看到什麼狀態。

做專題前：我能畫出資料流與元件表。